Vergleich von Systemverbesserungen X und Y

Es sollen zwei mögliche Verbesserungen (X und Y) an einem bestehenden Computersystem verglichen werden. Verwenden Sie Amdahls Gesetz und begründen Sie Ihre Antworten.

Verbesserung X:
Die Implementierung einer Ein/Ausgabe-Operation (zB lesen und schreiben von bzw. In Dateien) kann von 150 Taktzyklen auf 50 Taktzyklen pro Ein/Ausgabe-Operation beschleunigt werden. Die Ausführung der Ein/Ausgabe-Operationen beansprucht in der aktuellen Implementierung insgesamt 10 Prozent der gesamten Ausführungszeit eines typischen Programms.

Verbesserung Y:
Die Ausführung der trigonometrischen Funktion beansprucht in der aktuellen Implementierung insgesamt 40 Prozent der gesamten Ausführungszeit eines typischen Programms.

Problemstellung:
Um welchen Faktor muss die Ausführung einer einzelnen trigonometrischen Funktion beschleunigt werden, sodass Verbesserung Y eine größere Gesamtbeschleunigung erzielt als Verbesserung X?

Fehlertoleranz in Computersystemen – hoch-zuverlässige Systeme – F3of3 & F4of5

Es stehen zwei verschiedene Architektur-Entwürfe für ein IT-System zur Diskussion, das extrem hohen Zuverlässigkeitsanforderungen gerecht werden soll.

Der erste Entwurf **F3of 3**entspricht einem TMR (Triple Modular Redundancy, oberes Diagramm) System, das allerdings nur als funktionsfähig angesehen wird, wenn alle drei Komponenten **Comp_i sowie der Voter** fehlerfrei funktionieren.

Der zweite Entwurf **F4of5** umfasst ein FMR (Five-fold Modular Redundancy, unteres Diagramm) System, das als funktionsfähig angesehen wird, wenn mindestens vier der **fünf Komponenten Comp_i sowie der Voter** fehlerfrei funktionieren.

In beiden Entwürfen funktioniert der Voter mit der Wahrscheinlichkeit v. fehlerfrei und jede Komponente **Comp_i** funktioniert fehlerfrei mit der Wahrscheinlichkeit p.

Geben Sie in der folgenden Tabelle für $p = 0{,}98$ sowie für $p = 0{,}15$ mit jeweils $v = 0{,}96$ auf mindestens **fünf Nachkommastellen** die Wahrscheinlichkeiten an, dass **F3of3** bzw. **F4of5** mit den beschriebenen Zuverlässigkeits-Anforderungen korrekt funktioniere

Hamming-Abstand

Berechnung des Hamming-Abstands

Aus einem nicht-fehlertoleranten Code mit Hamming-Abstand b = 1 soll ein fehlertoleranter Hamming-Code erstellt werden, mit dem mindestens 5 Bit-Fehler pro Wort erkannt werden können und mindestens 3 Bit-Fehler pro Wort korrigiert werden können. Wie groß muss der Hamming-Abstand des fehlertoleranten Hamming-Codes mindestens sein?

Begründen Sie Ihre Antwort!

Hamming-Algorithmus – 27 Daten-Bits

Ein nicht-fehler tolerantes Code-Wort umfasst 27 Daten-Bits. Es soll gemäß dem Hamming-Algorithmus um Prüf-Bits ergänzt werden, so dass Ein-Bit-Fehler korrigiert werden können. Ergänzen Sie in der folgenden Tabelle die 27 Daten-Bits gemäß dem diskutierten Hamming-Algorithmus um die erforderliche Anzahl der Prüf-Bits und verwenden Sie **ungerade Parität**.

Erinnerung:

m + r + 1 <= 2^r

Bit-Position	Wert
1	?
2	?
3	1
4	?
5	0
6	1
7	1
8	?
9	0
10	1
11	1
12	1
13	1
14	0
15	0
16	?
17	1
18	0
19	1
20	1
21	0
22	1
23	1
24	0
25	1
26	0
27	1
28	1
29	0
30	0
31	0
32	?
33	0

Speicherhierarchie und Speicherzugriffszeiten

Ein Rechnersystem verfügt zusätzlich zum Hauptspeicher über 3 Cache-Hierarchien mit L1 (Level 1) Cache, L2 Cache und L3 Cache.

L1 und L2 Cache befinden sich auf dem CPU-Chip und Zugriffe auf L1 und L2 Cache werden parallel ausgeführt.

Zugriffe auf L3 Cache und Hauptspeicher werden sequentiell, d.h. nacheinander und erst nach dem Zugriff auf den vorherigen L1 und L2 Cache Ebenen, ausgeführt.

L1 Cache, L2 Cache, L3 Cache und Hauptspeicher weisen folgende Zugriffszeiten für Lesezugriffe auf:

- L1 Cache: 2 ns (Nanosekunden)
- L2 Cache: 5 ns
- L3 Cache: 30 ns
- Hauptspeicher: 100 ns

Mittels eines speziellen Benchmarks wurden folgende Trefferquoten (Cache Hits) bei Lesezugriffen gemessen:

- 20 Prozent aller Speicher-Lesezugriffe sind Cache-Misses (Fehlschläge) des L1 Caches.
- 16 Prozent aller Speicher-Lesezugriffe sind L2 Cache-Hits.
- 3,2 Prozent aller Speicher-Lesezugriffe sind L3 Cache-Hits

a) Mittlere Zugriffszeit für lesende Speicherzugriffe

Wie groß ist die mittlere Zugriffszeit für lesende Speicherzugriffe?

b) Optimierung der Speicherhierarchie

Wird die mittlere Zugriffszeit für lesende Speicherzugriffe verbessert oder verschlechtert, wenn:

1. Der L3 Cache entfernt wird
2. Ein deutlich schnellerer Hauptspeicher mit 48 ns Lesezugriffszeit verwendet wird?
   (Dabei bleiben die Werte für L1 und L2 Caches unverändert)

Sekundäre Speicher

Magnetische Festplatte – HD – Hard Drive

Eine ausgewählte Spur einer magnetischen Festplatte enthält 2^11 Sektoren der Größe 4096 Byte. Die vorhandene Festplatte besteht aus 5 übereinanderliegenden Scheiben. Wie groß ist die Kapazität des Zylinders der Festplatte, der die ausgewählte Spur beinhaltet?
Geben Sie das Ergebnis in MiB (Mega Binary Byte) an.

Hinweis: Die nebenstehende Graphik illustriert den schematischen Aufbau einer magnetischen Festplatte mit 4 Scheiben.

Flash Memory – SSD – Solid State Drive

a) Nach wievielen Jahren haben alle Speicherzellen eines SSD die maximale Anzahl von Lösch/Schreibvorgängen erreicht, wenn die TBW (Total Bytes Written) des SSD mit 10^{14} angegeben ist und
pro Tag 2^{36} Byte Daten auf das SSD geschrieben werden? Neu zu schreibende Daten überschreiben dabei
jeweils die ältesten Daten. Gehen Sie von 365 Tagen pro Jahr aus.

b) Ein ausgewähltes SSD beinhaltet keine statischen Daten und hat eine Kapazität von 512 GiB, die zu 50% belegt ist. Es wird Wear Leveling verwendet. Von welcher Lebensdauer kann ausgegangen werden, wenn:

1. Jede gespeicherte Datei mit dynamischen Daten (bei gleichbleibender Größe) im Mittel alle 45 Minuten aktualisiert wird?
2. Die maximale Anzahl von Lösch/Schreibvorgängen für jeden Speicherblock 24.000 beträgt?

Geben Sie das Ergebnis in Jahren an (365 Tage pro Jahr)

Arbeitsweise eines Flash Memory

Ein Flash Memory (z.B. ein Solid State Drive / SSD) verfügt über eine **Speicherkapazität von 128 GiB** mit einer Block-Größe **von 2 MiB**. Vom Hersteller wird eine **maximale Anzahl von 4096 Lösch/Schreib-Zyklen pro Speicherzelle angegeben**. Reserve-Blöcke („Spare-Blocks“) sind nicht vorhanden.

Aktuell sind **1024 Dateien mit dynamischen Daten** gespeichert. Jede Datei mit dynamischen Daten wird einmal pro Stunde aktualisiert. Jede Datei mit dynamischen Daten belegt 4 MiB.

Es wird **Dynamisches Wear Leveling** angewandt. Die Lebensdauer des Flash Memory soll mindestens 8 Jahre bei 365 Tagen pro Jahr betragen.

Bestimmen Sie die maximale Größe des Speicherbereichs des Flash Memory, der von Dateien mit statischen Daten belegt werden kann. Geben Sie diese maximale Größe in „Anzahl von Blöcken“ an.

Erinnerung „Lebensdauer“: Anzahl der maximalen möglichen Lösch/Schreibzyklen / Anzahl der durchgeführten Lösch/Schreibzyklen pro Zeiteinheit

Synchrone Bus-Systeme

Das folgende Diagramm sowie die zugehörige Tabelle beschreiben die Eigenschaften und das Verhalten eines synchronen Busses bei einem lesenden Speicherzugriff. Weitere zeitliche Rahmenbedingungen existieren für den betrachteten Bus nicht. Begründen Sie Ihre Antworten auf die folgenden Fragen!

Symbol	**Parameter**	**Min**	**Max**	**Unit**
Tad	Address output delay		4	nsec
Tds	Data setup time prior to falling edge of clock	?		nsec

Hinweis: 1 nsec = 1 Nanosekunde = 10^−9 Sekunden

---

a) Der Bus ist mit 80 MHz getaktet. Bestimmen Sie den Wert von TDS, so dass:

1. Der lesende Speicherzugriff in drei Taktzyklen erfolgen kann
2. Speicherkomponenten verwendet werden können, die 22 nsec von der Bekanntgabe der Speicheradresse über den Bus bis zum Legen der Daten auf den Bus benötigen

b) Betrieb mit 62,5 MHz Taktfrequenz

Wir nehmen nun eine Bustaktung von 62,5 MHz an und betrachten den Parameter TDS (Data Setup Time) mit einem Wert von 5 ns. Wie viele Buszyklen sind zur Bearbeitung eines lesenden Speicherzugriffs mindestens erforderlich, wenn:

1. Die Speicherkomponenten 32 ns von der Adressbekanntgabe bis zum Datenbereitstellen benötigen
2. Die Notwendigkeit von Wait-Zyklen zu berücksichtigen ist

Superskalarität mit heterogenen Ausführungseinheiten und Instruktionsabhängigkeiten

Moderne leistungsstarke Computerarchitekturen verfügen über mehrere Ausführungseinheiten, die jeweils spezielle Instruktionstypen ausführen (siehe anbei Ausschnitt aus Cortex-A72 Prozessor Core[1]).

In einem Code-Segment mit den 12 Instruktionen L_1 bis L_12, die in der sequentiellen Reihenfolge
L_1, L_2, L_3, ..., L_11, L_12 im Programm-Code stehen, bestehen die folgenden 11 RAW-Abhängigkeiten (*Read After Write*):

L1→L2,L1→L3,L1→L6,L1→L9,L2→L3,L2→L5,L4→L6,L1→L2,L1→L3,L1→L6,L1→L9,L2→L3,L2→L5,L4→L6,L5→L6,L7→L8,L7→L10,L10→L11.L5→L6,L7→L8,L7→L10,L10→L11.

Jede Instruktion benötigt genau einen Taktzyklus zur vollständigen Bearbeitung.

Die Instruktionen L_6, L_7, L_10 und L_12 sind trigonometrische Operationen.

Die Instruktionen L_1, L_2, L_3, L_4, L_5, L_8, L_9 und L_11 sind Additionen oder Multiplikationen.

a) Wie viele Taktzyklen werden zur Ausführung dieses Code-Segments auf einem Prozessor mit den drei Ausführungseinheiten AE1, AE2 und AE3 mindestens benötigt, wenn die Ausführungsreihenfolge
L_1, L_2, ..., L_12 nicht modifiziert werden darf?

Dabei können auf AE1 und AE2 nur Additionen oder Multiplikationen zur Ausführung kommen. Auf AE3 können nur trigonometrische Operationen zur Ausführung kommen.

Geben Sie in der folgenden Tabelle eine entsprechende Ausführungsreihenfolge der Instruktionen mit minimaler Ausführungszeit auf den drei Ausführungseinheiten an. Jede Instruktion muss möglichst frühzeitig zur Ausführung kommen

Taktzyklus	AEI (ADD & MULT)	AE2 (ADD & MULT)	AE3 (Trig. OP)
1			
2			
3			
4			
5			
6			

7			
8			
9			
10			
11			
12			

b) Wie viele Taktzyklen werden zur Ausführung dieses Code-Segments auf einem Prozessor mit den drei Ausführungseinheiten AE1, AE2 und AE3 mindestens benötigt, wenn die Ausführungsreihenfolge

I_1, I_2, ..., I_12 verändert werden darf?

Dabei können auf AE1 und AE2 nur Additionen oder Multiplikation zur Ausführung kommen. Auf AE3 können nur trigonometrische Operationen zur Ausführung kommen.

Geben Sie in der folgenden Tabelle eine entsprechende Ausführungsreihenfolge der Instruktionen mit minimaler Ausführungszeit auf den drei Ausführungseinheiten an. Jede Instruktion muss möglichst frühzeitig zur Ausführung kommen.

Taktzyklus	AEI (ADD & MULT)	AE2 (ADD & MULT)	AE3 (Trig. OP)
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			

Erweiterung der n-Byte Operanden auf 32 Bit in den Mic-Architekturen

Hinweise: Zahlendarstellung in den Mie-Architekturen folgt dem 2er Komplement

a) In Register MBR bzw. MBRU der Mic1-Architektur befindet sich der Wert **OxBF** (hexadezimal angegebene). Um diesen Wert vom Register zur ALU zu transportieren, muss er auf 32 Bit erweitert werden. Geben Sie in folgender Tabelle binär die Belegung der 32 Bit an, wenn es sich um einen **Unsigned Value** (Nicht-Vorzeichenbehafteter Wert) handelt und Bit-Position 0 das niederwertigste Bit der Erweiterung auf 32 Bit beinhaltet.

Bit Pos.	**31**	**30**	**29**	**28**	**27**	**26**	**25**	**24**	**23**	**22**	**21**	**20**	**19**	**18**	**17**	**16**
Bit Wert																

Bit Pos.	**15**	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Bit Wert																

b) In Register MBR2 bzw. MBR2U der Mic1-Architektur befindet sich der Wert **Ox8F7A** (hexadezimal angegebene). Um diesen Wert vom Register zur ALU zu transportieren, muss er auf 32 Bit erweitert werden. Geben Sie in folgender Tabelle binär die Belegung der 32 Bit an, wenn es sich um einen **Signed Value** (Vorzeichenbehafteter Wert) handelt und Bit-Position 0 das niederwertigste Bit der Erweiterung auf 32 Bit beinhaltet.

Bit Pos.	**31**	**30**	**29**	**28**	**27**	**26**	**25**	**24**	**23**	**22**	**21**	**20**	**19**	**18**	**17**	**16**
Bit Wert																

Bit Pos.	**15**	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Bit Wert																

c) In Register MBR bzw. MBRU der Mic1-Architektur befindet sich der Wert **Ox73** (hexadezimal angegebene). Um diesen Wert vom Register zur ALU zu transportieren, muss er auf 32 Bit erweitert werden. Geben Sie in folgender Tabelle binär die Belegung der 32 Bit an, wenn es sich um einen **Signed Value** (Vorzeichenbehafteter Wert) handelt und Bit-Position 0 das niederwertigste Bit der Erweiterung auf 32 Bit beinhaltet.

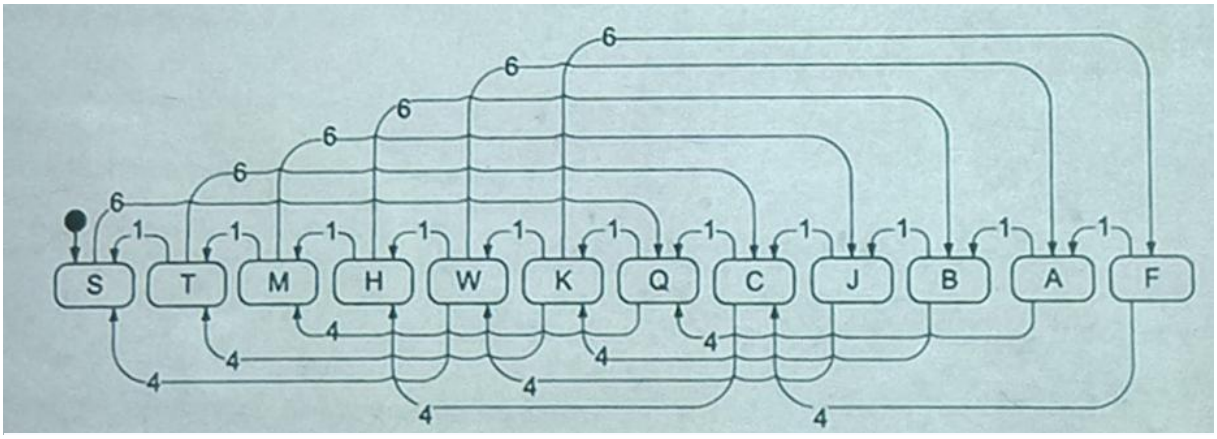
Bit Pos.	**31**	**30**	**29**	**28**	**27**	**26**	**25**	**24**	**23**	**22**	**21**	**20**	**19**	**18**	**17**	**16**
Bit Wert																

Bit Pos.	**15**	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Bit Wert																

IFU – Instruction Fetch Unit

Die IFU (Instruction Fetch Unit) der in der Lehrveranstaltung diskutierten Mic-2 Architektur beinhaltet ein Schieberegister (Shift Register).

Der folgende Endliche Automat (FSM – Finite State Machine) veranschaulicht die möglichen Zustände einer weiter entwickelten IFU mit einem Schieberegister einer anderen Kapazität. Die Zahl eines Zustandsübergangs, der als Pfeil von rechts nach links führt, entspricht der Entnahme von y Bytes aus dem Schieberegister in einem Taktzyklus. Die Zahl eines Zustandsübergangs, der als Pfeil von links nach rechts führt, entspricht dem Hinzufügen von y Bytes in das Schieberegister in einem Taktzyklus.
Der Zustand S ist der Startzustand des endlichen Automaten, in dem das Schieberegister der IFU leer ist, also kein Byte enthält.



a) Wie groß ist die Kapazität des Schieberegisters, dessen Füllzustand durch die FSM beschrieben wird?
(Mit anderen Worten: Wie viele Bytes kann das Schieberegister maximal aufnehmen?)

b) Wie viele Bytes können in einem Taktzyklus aus dem Schieberegister entnommen werden?

c) Wie viele Bytes können in einem Taktzyklus in das Schieberegister hinzugefügt werden?

d) Wie viele Bytes muss das Schieberegister <u>mindestens</u> aufnehmen können, damit die im Endlichen Automaten skizzierte Vorgehensweise funktioniert, aber die Kapazität des Schieberegisters möglichst klein sein soll?
Mit anderen Worten: Ist eine Verkleinerung des Schieberegisters möglich, ohne die korrekte Funktionalität zu beeinträchtigen?
Geben Sie die Namen der Zustände an, die eliminiert werden können, ohne die korrekte Funktionalität zu beeinträchtigen.
Begründen Sie Ihre Antwort!

Adressierung

A) Adressierungsarten: Ein Rechensystem mit einem Akkumulator-Register sowie weiteren Registern R1, R2 und R3 besitzt augenblicklich folgende Werte an den jeweiligen Speicheradressen sowie in den angegebenen Registern.

Geben Sie die Werte an, die – jeweils abhängig von dem Modus der Adressierung – in den Akkumulator geladen werden.

a) LOAD IMMEDIATE 130
b) LOAD DIRECT 130
c) LOAD INDIRECT 120
d) LOAD REGISTER-MODE R2
e) LOAD INDIRECT-REGISTER-MODE R3
f) LOAD INDEXED-MODE 40 R2
g) LOAD BASED-INDEXED-MODE R1 R2

Speicheradresse	Speicherinhalt
120	140
130	120
140	180
150	130

Register	Registerinhalt
R1	20
R2	110
R3	140

Cache-Speicher

TAG	LINE	WORD/BYTE
35 Bit	20 Bit	9 Bit

Der Cache eines Computers verfügt über eine Kapazität von 2^{33} Byte. Der Computer verweist die nebenstehende Speicheradresse in der gegebenen Struktur. Beantworten Sie folgende Fragen und begründen Sie Ihre Antworten.

a) Wie viele Byte befinden sich in einer Cache-Zelle?

b) Welche Größe – in Anzahl von Bits – hat die Speicheradresse?

c) Wie viele Cache-Zellen befinden sich in dem Cache?

d) Handelt es sich um einen direkt abgebildeten Cache oder um einen n-Wege teil-assoziativen Cache? Falls es sich um einen n-Wege teil-assoziativen Cache handelt, geben Sie die Größe n an.

Teilassoziativer n-Weg-Cache

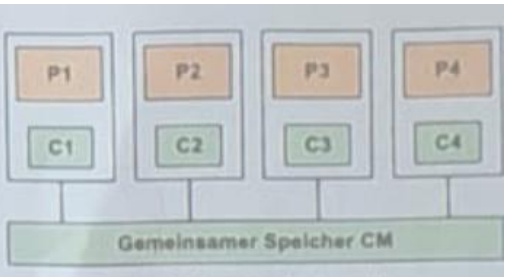
Ein teilassoziativer n-Weg-Cache der Kapazität 1 MByte (2^{20} Byte) verwaltet Cache-Zeilen der Größe 128 Byte. Das Rechenersystem verwendet 32-Bit.

Welche Menge (auch Eintrag oder Entry genannt) muss anhand des Tags Speicheradresse 0x3476F5A8 im Cache befindet? Geben Sie den gesuchten Wert – d.h. die Nummer der Menge – als Dezimalwert an.

Anmerkung: Die zu durchsuchende Menge wird über den Line-Eintrag der Speicheradresse identifiziert:

Tag	**LINE**	**WORDBYTE**

1 Zustände des Cache-Kohärenzprotokolls MESI korrekt?



Ein paralleles Rechnersystem mit vier Prozessoren (**P1**, **P2**, **P3**, **P4**) und vier lokalen Caches (**C1**, **C2**, **C3**, **C4**) sowie einem gemeinsamen Speicher **CM** (Common Memory) arbeitet nach dem MESI Protokoll. Im Gemeinsamen Speicher (CM) ist die jeweilige Cache-Zeile gültig (**V** – valid) oder ungültig (**I** – invalid). Geben Sie in den folgenden Teilaufgaben an, ob für eine gegebene Cache-Zeile **CLY** die jeweiligen Zustände möglich sind und begründen Sie Ihre Antworten!

a) MESI-Zustände der Cache-Zeile **CLY** zum Zeitpunkt **T1**.

Zustand CLY: C1	Zustand CLY: C2	Zustand CLY: C3	Zustand CLY: C4	Zustand im CM
I	M	I	E	V

b) MESI-Zustände der Cache-Zeile **CLY** zum Zeitpunkt **T2**.

Zustand CLY: C1	Zustand CLY: C2	Zustand CLY: C3	Zustand CLY: C4	Zustand im CM
I	S	S	E	I

c) MESI-Zustände der Cache-Zeile **CLY** zum Zeitpunkt **T3**.

Zustand CLY: C1	Zustand CLY: C2	Zustand CLY: C3	Zustand CLY: C4	Zustand im CM
S	S	I	I	V

d) MESI-Zustände der Cache-Zeile **CLY** zum Zeitpunkt **T4**.

Zustand CLY: C1	Zustand CLY: C2	Zustand CLY: C3	Zustand CLY: C4	Zustand im CM
I	I	M	I	V

			I	?

b) Welche gültigen Zustände kann die Cache-Zeile **CLY** des Caches **C1** zum Zeitpunkt **T2** annehmen, wenn von einem kann und die Cache-Zeilen **CLY** der Caches **C2**, **C3** und **C**

Zustand CLY: C1	Zustand CLY: C2	Zustand CLY: C3	Zustand CLY: C4	Zustand im CM
?	S	S	S	?

c) Welche gültigen Zustände kann die Cache-Zeile **CLY** des C von einem korrekten MESI-Protokoll ausgegangen werden **C2** und **C4** die folgenden MESI-Zustände besitzen?

Zustand CLY: C1	Zustand CLY: C2	Zustand CLY: C3	Zustand CLY: C4	Zustand im CM
I	I	?	E	?

f) Welche gültigen Zustände kann die Cache-Zeile **CLY** des Cac von einem korrekten MESI-Protokoll ausgegangen werden ka **C2** und **C4** die folgenden MESI-Zustände besitzen?

Zustand CLY: C1	Zustand CLY: C2	Zustand CLY: C3	Zustand CLY: C4	Zustand im CM
S	I	?	I	?

Welche gültigen Zustände kann die Cache-Zeile **CLY** des von einem korrekten MESI-Protokoll ausgegangen werden **C3** und **C4** die folgenden MESI-Zustände besitzen?

Zustand CLY: C1	Zustand CLY: C2	Zustand CLY: C3	Zustand CLY: C4	Zustand im CM
?	I	M	I	?

Ausführungszeit eines ISA-UVM Programms auf einer Mic-1 Architektur

Welche Ausführungszeit benötigt das folgende in der ISA der UVM gegebene Programm auf einer mit 156,25 MHz getakteten Mic-1 Architektur?
Geben Sie das Ergebnis in Nanosekunden an und liefern Sie eine detaillierte Berechnung des Ergebnisses.

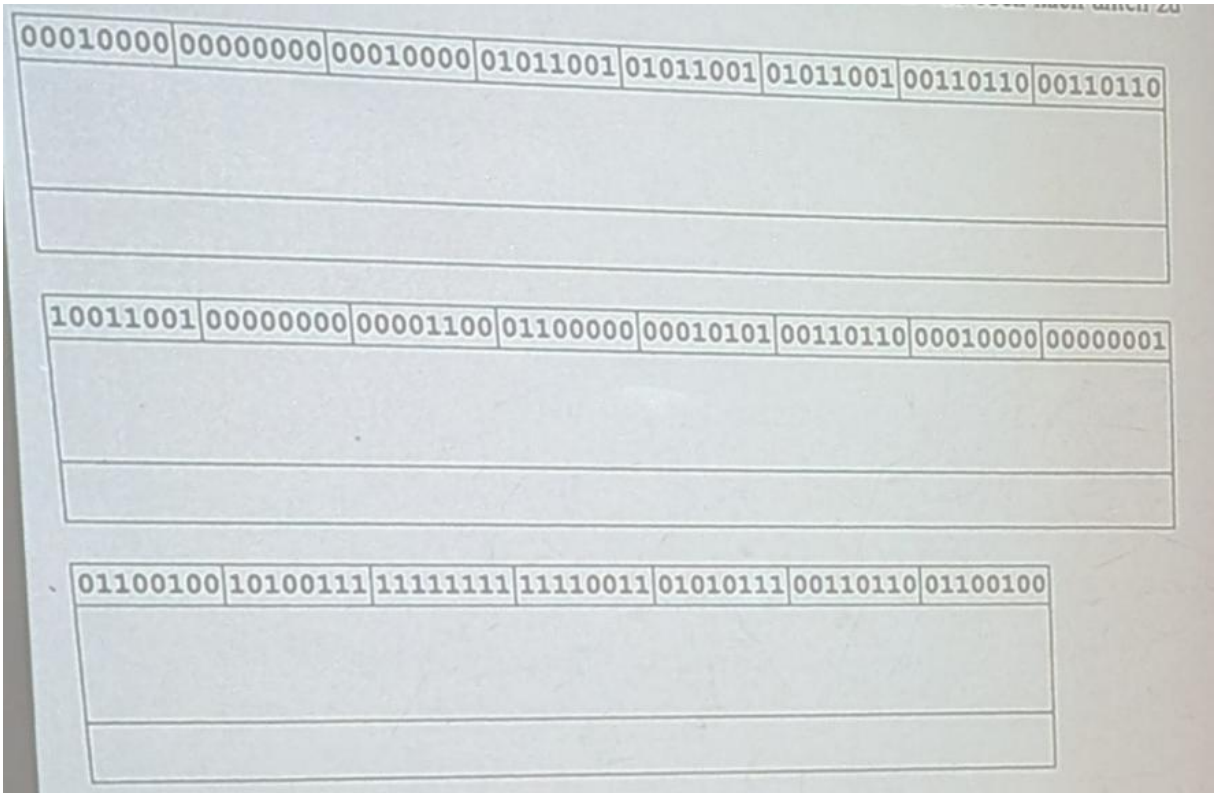
Hinweis: 1 Nanosekunde = 10^−9 Sekunden10−9Sekunden, 1 MHz = 10^6 Hz

IJVM – Programm:

```
ILOAD P
ILOAD Q
DUP
IADD
NOP
SWAP
ISUB
ISTORE Z
```

ISA-IJVM Programm Dekodierung

Analysieren Sie das folgende ISA-IJVM-Programm, das in Binär-Code angegeben ist. Der Binär-Code ist byte-weise in jeweils 8 Bit angegeben und ist von links nach rechts sowie von oben nach unten zu lesen



00010000	00000000	00010000	01011001	01011001	01011001	00110110	00110110

10011001	00000000	00001100	01100000	00010101	00110110	00010000	00000001

01100100	10100111	11111111	11110011	01010111	00110110	01100100

Dabei liegen im lokalen Variablennahmen an den folgenden - im Dezimalsystem gegebenen - Wort-Adressen die Werte der folgenden Variablen und wir gehen von nicht-negativen Integer-Werten der Variablen aus:

Wort-Adr.	**10**	**21**	**36**	**54**	**59**	**64**	**80**	**89**	**100**	**150**
Variable	**L**	**J**	**C**	**K**	**A**	**S**	**X**	**Z**	**Q**	**P**

Hinweis: Berücksichtigen Sie, dass Sprungadressen auch negative ganzzählige Werte sein können, die im Zweier-Komplement dargestellt werden.

a) Erstellen Sie die für Menschen besser lesbare Form dieses ISA-UVM-Programms mit den bekannten Mnemonics wie IADD, ILOAD, ISTORE, etc.

b) Wie groß ist die maximal erreichte Höhe des Operanden-Stapels und nach dem wievielten Byte des Hex-Byte-Codes wird diese Höhe erreicht?

c) Welche Werte stehen nach Programmausführung in den Variablen K, C und Q, wenn alle Variablen bei Programmstart den Dezimalwert 8 besitzen?

d) Geben Sie in Java/C#/C++ - ähnlicher Notation an, was dieses Programm berechnet.